

Operating System (CSE231)

Section-B

End-semester Examination Rubrics

Total Marks: 95

Time: 2 hrs

1. (5 points) Consider the following processes in Table 1 with their CPU burst times (in milliseconds):

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Table 1: Process Arrival and Burst Times

Using a non-preemptive Shortest Job First (SJF) scheduling algorithm, calculate the average waiting time for the four processes.

[Answer]. 1 Mark: Correctly identifying that P1 runs first (handling the "Time 0" constraint). Calculating Waiting Time ($Start - Arrival$) for each process, then average (4 marks): P1 = 0; P2 = 7; P3 = 15; P4 = 9; Average waiting time: 7.75 ms

$$TAT = \text{Finish Time} - \text{Arrival Time}$$

$$TAT \text{ of P1} = 8 - 0 = 8$$

$$TAT \text{ of P2} = 12 - 1 = 11$$

$$TAT \text{ of P3} = 17 - 2 = 15$$

$$TAT \text{ of P4} = 26 - 3 = 23$$

$$\text{Avg TAT} = \frac{8 + 11 + 15 + 23}{4} = \frac{57}{4} = 14.25 \text{ ms}$$

$$RT = \text{Start Time} - \text{Arrival Time}$$

Average Response Time

$$\text{Avg RT} = \frac{0 + 7 + 9 + 15}{4} = \frac{31}{4} = 7.75 \text{ ms}$$

Consider either of them as the waiting time.

2. (5 points) A process has access to 3 frames. The page reference string is: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. Using the First-In, First-Out (FIFO) page replacement algorithm, what is the total number of page faults and the hit rate? Show the steps.

[Answer].Demonstrating the FIFO replacement logic. 2 Marks: Correctly handling the first 10-12 replacements (showing the "oldest" gets replaced). 1 Mark: Correctly identifying the "Hits" (not changing frames during a hit). 1 mark for the correct total number of page faults and 1 mark for using the correct hit rate formula and the right answer.

Ref	Frame 1	Frame 2	Frame 3	Status	Explanation
7	7			Fault	Empty frame, load 7
0	7	0		Fault	Empty frame, load 0
1	7	0	1	Fault	Empty frame, load 1 (Frames full)
2	2	0	1	Fault	Replace 7 (Oldest)
0	2	0	1	Hit	0 is already in Frame 2
3	2	3	1	Fault	Replace 0 (Next oldest)
0	2	3	0	Fault	Replace 1 (Next oldest)
4	4	3	0	Fault	Replace 2 (Next oldest)
2	4	2	0	Fault	Replace 3
3	4	2	3	Fault	Replace 0
0	0	2	3	Fault	Replace 4
3	0	2	3	Hit	3 is already in Frame 3
2	0	2	3	Hit	2 is already in Frame 2
1	0	1	3	Fault	Replace 2
2	0	1	2	Fault	Replace 3
0	0	1	2	Hit	0 is already in Frame 1
1	0	1	2	Hit	1 is already in Frame 2
7	7	1	2	Fault	Replace 0
0	7	0	2	Fault	Replace 1
1	7	0	1	Fault	Replace 2

Table 2: FIFO Page Replacement Step-by-Step

- **Total Page Faults: 15**
- **Total Hits: 5**
- **Hit Rate: $\frac{5}{20} = 25\%$**

3. (10 points) Consider a computer system featuring a 32-bit logical address space and a page size of 4 KB. The system utilizes 4-byte Page Table Entries (PTEs). Calculate the total memory required for the page table with a standard single-level paging

scheme, and concisely explain why requiring this amount of contiguous memory is problematic for an operating system.

To resolve this issue, design a two-level paging architecture for this system such that the Outer Page Directory fits exactly into a single page frame. Based on this constraint, determine the specific partitioning of the 32-bit logical address into the Outer Page Number (p_1), Inner Page Number (p_2), and Page Offset (d). Show all calculations used to derive the bit counts for each section.

[Answer]. (4 Marks) Logical Address Space: 32-bit $\rightarrow 2^{32}$ bytes. Page Size: 4 KB $\rightarrow 2^{12}$ bytes. PTE Size: 4 Bytes.

$$\text{Number of Pages} = \frac{\text{Logical Address Space}}{\text{Page Size}} = \frac{2^{32}}{2^{12}} = 2^{20} \text{ pages}$$

The Page Table must have one entry per page:

$$\text{Page Table Size} = (\text{Number of Pages}) \times (\text{PTE Size})$$

$$\text{Page Table Size} = 2^{20} \times 4 \text{ bytes} = 2^{20} \times 2^2 \text{ bytes} = 2^{22} \text{ bytes}$$

Result: The total memory required is 4 MB. Correctly deriving 2^{20} pages and converting to 4 MB

(1 Mark) Mentioning the difficulty of finding contiguous memory or external fragmentation.

(1 mark) Calculate Offset (d): The offset is determined strictly by the page size. Page Size = 4 KB = 2^{12} bytes. $d = 12$ bits .

(3 Marks) Calculate Outer Page Number (p_1) The problem states the Outer Page Directory must fit exactly into one page frame. Frame Size = 4 KB (2^{12} bytes). Entry Size (PTE) = 4 bytes (2^2 bytes). We calculate how many entries can fit in one frame:

$$\text{Max Entries} = \frac{\text{Frame Size}}{\text{Entry Size}} = \frac{2^{12}}{2^2} = 2^{10} \text{ entries}$$

Since the Outer Directory has 2^{10} entries, we need 10 bits to address them. $p_1 = 10$ bits.

(1 Marks) Calculate Inner Page Number (p_2) The total logical address is 32 bits. We subtract the bits used for the offset and the outer page number to find the remainder.

$$\text{Total Bits} = p_1 + p_2 + d$$

$$32 = 10 + p_2 + 12$$

$$p_2 = 32 - 22$$

$$p_2 = 10 \text{ bits.}$$

4. (6 points) A process installs a custom signal handler for SIGINT (Ctrl-C) that prints the message "Interrupt received". After setting the handler, the process executes a long computation inside a loop. During this computation, the user presses Ctrl-C twice in quick succession. Assume the following.
 1. SIGINT is not blocked or ignored.
 2. The handler runs quickly (it just prints a message).

3. No other signals are sent.
4. The program does not change the handler after installing it.

Answer the following.

- (a) (2 points) What output will the user see and why?
- (b) (2 points) After the two Ctrl-C presses, does the program continue running or terminate? Explain.
- (c) (2 points) What would happen if the program restored the default action for SIGINT instead of installing a handler?

[Answer]. a) The message “Interrupt received” will be printed twice by the handler. Each Ctrl-C sends a SIGINT, and since the handler runs quickly and returns, both signals are delivered.

b) The program continues running after the handler finishes. The reason for that is as follows.

- The handler overrides the default action.
- The default action of SIGINT is process termination, but the handler replaces it.
- Since the handler simply prints a message and returns, normal execution resumes.

c) If the program used the default action for SIGINT, then

- The first Ctrl-C immediately terminates the process.
- The second press never occurs because the program is already dead.

With the default action, the process exits immediately on receiving the first SIGINT.

5. (10 points) Consider a file descriptor `fd` associated with a regular file. The file size is 30 bytes. The current file offset is 0. Execute the following sequence of operations.

- `read(fd, buffer, 5);`
- `lseek(fd, 10, SEEK_CUR);`
- `write(fd, buffer, 5);`
- `lseek(fd, 10, SEEK_END);`
- `write(fd, buffer, 10);`
- `lseek(fd, -5, SEEK_END);`
- `read(fd, buffer, 20);`

- (a) (5 points) What is the numerical value of the file offset after each of the 7 operations? Show your calculation logic.
- (b) (3 points) What is the final size of the file (in bytes) after step 5?
- (c) (2 points) In step 7, the code requests to read 20 bytes. How many bytes are actually returned by this read call, and why?

[Answer]. Part (a) (5 marks)

Step, Operation, Logic & Calculation, New Offset

1, read(fd, buff, 5), Reads 5 bytes from current offset (0). Calculation: $0+5 = 5$

2, lseek(fd, 10, SEEK_CUR), Moves offset 10 bytes from the current position (5).

Calculation: $5+10 = 15$

3, write(fd, buff, 5), Writes 5 bytes starting at offset 15. Calculation: $15+5 = 20$

4, lseek(fd, 10, SEEK_END), Moves offset to specific position relative to end of file.

Current file size is 30. Calculation: $30+10 = 40$

5, write(fd, buff, 10), Writes 10 bytes starting at offset 40. Calculation: $40+10 = 50$

6, lseek(fd, -5, SEEK_END), Moves offset relative to the new end of file. Current file size is now 50 (updated in step 5). Calculation: $50 + (-5) = 45$

7, read(fd, buff, 20), Reads 20 bytes (or as many as available). The offset advances by the number of bytes actually read. Available bytes = $50(\text{size}) - 45(\text{current}) = 5$.

Calculation: $45+5 = 50$

(Part b) 3 Marks for the correct answer: 50. Partial Credit (1 Mark): If the student answers 49 (off-by-one error) or 40 (forgot to add the length of the written buffer). 0 Marks: If the student answers 30 (failed to understand file extension).

(Part c) 1 Mark for the correct value: 5. **1 Mark** for the justification: Must mention End of File (EOF) or calculation showing only 5 bytes were remaining.

6. (10 points) Consider a filesystem managing a disk with block size 2^b bytes, and disk block addresses of 2^a bytes. The inode of a file contains n direct blocks, one single indirect block, one double indirect block, and one triple indirect block. What is the maximum size of a file (in bytes) that can be stored in this filesystem? Explain your answer. Assume that the indirect blocks only store a sequence of disk addresses, and no other metadata.

[Answer]. (2 mark)

Block Size: 2^b bytes

Address Size: 2^a bytes

Number of addresses per block (N):

$$N = \frac{\text{Block Size}}{\text{Address Size}} = \frac{2^b}{2^a} = 2^{b-a}$$

(1 mark) Total size:

$$S_{\text{direct}} = n \times 2^b$$

(1 mark) Total size:

$$S_{\text{single}} = N \times 2^b = 2^{b-a} \times 2^b = 2^{2b-a}$$

(2 mark) Number of data blocks: $N \times N = N^2$

Total size:

$$S_{\text{double}} = N^2 \times 2^b = (2^{b-a})^2 \times 2^b = 2^{2b-2a} \times 2^b = 2^{3b-2a}$$

(2 mark) Number of data blocks: $N \times N \times N = N^3$

Total size:

$$S_{triple} = N^3 \times 2^b = (2^{b-a})^3 \times 2^b = 2^{3b-3a} \times 2^b = 2^{4b-3a}$$

(1 mark)

$$\text{Max Size} = S_{direct} + S_{single} + S_{double} + S_{triple}$$

(1 mark)

$$\text{Max Size} = 2^b \cdot [n + 2^{b-a} + 2^{2(b-a)} + 2^{3(b-a)}] \text{ bytes}$$

or

$$\text{Max Size} = 2^b \cdot [n + N + N^2 + N^3] \text{ bytes}$$

7. (20 points) Two threads, A and B, must reach a meeting point before both are allowed to proceed. Example output is as follows (order of arrival can vary, but both “passed” lines must appear only after both arrived).

```
A reached rendezvous
B reached rendezvous
A passed rendezvous
B passed rendezvous
```

The following implementation attempts to achieve this but contains synchronization bugs.

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int arrived = 0;    // number of threads that reached the rendezvous

void rendezvous() {
    pthread_mutex_lock(&m);
    arrived++;
    if (arrived < 2) {
        pthread_cond_wait(&cv, &m);
    }
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&m);
}

void *threadA(void *arg) {
    printf("A reached rendezvous\n");
    rendezvous();
    printf("A passed rendezvous\n");
    return NULL;
}
```

```

void *threadB(void *arg) {
    printf("B reached rendezvous\n");
    rendezvous();
    printf("B passed rendezvous\n");
    return NULL;
}

```

Tasks

- (a) (6 points) Identify the synchronization errors or potential race conditions in the code. Explain why they are incorrect and what wrong behaviour may occur.
- (b) (8 points) Explain how condition variables must be used correctly in such a scenarios.
- (c) (6 points) Rewrite the corrected version of the rendezvous() function (you do not need to rewrite the full program).

[Answer]

1. Synchronization errors / potential race conditions

- (a) Using if instead of while before pthread_cond_wait(). Condition variables may experience spurious wakeups, or wakeups caused by unrelated signals, or the awakened thread might re-check the condition and find it false. Using if will cause a thread to continue without ensuring the condition is true. This can lead to a thread passing the rendezvous early before both threads have arrived, a lost wakeup, causing a deadlock.
- (b) Incorrect use of pthread_cond_signal(). The code signals the condition variable unconditionally, even when no thread is waiting or when the waiting condition is already satisfied. This may wake up a thread before it begins waiting, causing the thread to wait forever, cause lost wakeups, and cause threads to pass the rendezvous incorrectly.
- (c) Both threads may attempt to wait or signal improperly. The second thread must not wait, otherwise both may wait and deadlock. The first thread must wait, otherwise it may pass too early. The buggy code does not enforce this.

2. How condition variables must be used correctly

- (a) Every pthread_cond_wait() must be inside a while loop, not if. This ensures the condition is rechecked after waking.
- (b) The condition variable should be signaled only when the condition becomes true.
- (c) The mutex must be held when checking the condition, when calling wait(), and when signaling.
- (d) The waiting thread must re-acquire the mutex after waking and recheck the condition.

3. Corrected rendezvous() function

```

void rendezvous() {
    pthread_mutex_lock(&m);
    arrived++;
}

```

```

        if (arrived == 2) {
            // Second thread arrives: wake the first thread
            pthread_cond_signal(&cv);
        } else {
            // First thread arrives: wait for the second
            while (arrived < 2)
                pthread_cond_wait(&cv, &m);
        }
        pthread_mutex_unlock(&m);
    }
}

```

8. (15 points) Three threads A, B, and C execute the following actions.

- Thread A must print “A1” then “A2”.
- Thread B must print “B”.
- Thread C must print “C”.

The required ordering is as follows.

```

A1 must print before B
B must print before C
C must print before A2

```

There are no constraints between A1 and A2 other than the one above, and no constraints between B and C other than the ones specified. Using only semaphores, write synchronization code that ensures the above ordering is always respected. You may declare and initialize any semaphores needed, but clearly explain the role of each semaphore. Write only the semaphore initialization code and the code inside each thread (A, B, and C). Do not write code for thread creation or any other code apart from the one asked for.

[Answer] 3 points for semaphore initialization, 3 points for explanation of the role of each semaphore and 3 points each for each thread creation code.

```

sem_t semAldone; // A1 printed, wait for B to run
sem_t semBdone; // B printed, wait for C to run
sem_t semCdone; // C printed, wait for A2 to run
sem_init(&semAldone, 0, 0);
sem_init(&semBdone, 0, 0);
sem_init(&semCdone, 0, 0);

```

Thread A

```

// Thread A
printf("A1\n");
sem_post(&semAldone); // allow B to run
sem_wait(&semCdone); // wait for C before printing A2
printf("A2\n");

```

Thread B

```

// Thread B
sem_wait(&semAldone);           // must wait for A1
printf("B\n");
sem_post(&semBdone);           // allow C

```

Thread C

```

// Thread C
sem_wait(&semBdone);           // must wait for B
printf("C\n");
sem_post(&semCdone);           // allow A2

```

9. (14 points) Consider the following C program using threads and mutex, but with an intentionally incorrect critical section. Assume `printf()` prints atomically.

```

#include <pthread.h>
#include <stdio.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *threadA(void *arg) {
    pthread_mutex_lock(&m);
    counter += 1;
    pthread_mutex_unlock(&m);
    printf("A:%d ", counter);
    return NULL;
}

void *threadB(void *arg) {
    pthread_mutex_lock(&m);
    counter += 2;
    pthread_mutex_unlock(&m);
    printf("B:%d ", counter);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, threadA, NULL);
    pthread_create(&t2, NULL, threadB, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}

```

Tasks

1. List every possible valid output that this program can produce.
2. Explain briefly why each output is possible under some thread interleaving.

3. Explain why the mutex does not prevent all race conditions in this program, even though both threads use it.

[Answer]. 8 points for listing all the 4 valid outputs. 4 points for explaining the output using thread interleaving, and 2 points for explaining why mutex does not prevent all the race conditions.

The updates to counter are protected by the mutex, but the use of counter in printf() is outside the mutex. Thus, the final value seen by each thread during printing is not synchronized.

1. A runs first completely, then B. Output is A:1 B:3.
2. B runs first completely, then A. Output is B:2 A:3.
3. A increments $\rightarrow$ B increments $\rightarrow$ A prints $\rightarrow$ B prints. Output is A:3 B:3.
4. B increments $\rightarrow$ A increments $\rightarrow$ B prints $\rightarrow$ A prints. Output is B:3 A:3.

No other output is possible because counter after both updates must be 3, only the first print can show a partial counter value (1 or 2), and the second print must always show 3.